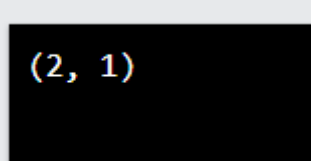


## Heapq internally uses less than operator

---

```
x = divmod(5, 2)
```

```
print(x)
```

A terminal window with a black background and a light gray title bar. The text "(2, 1)" is displayed in a yellow monospaced font.

```
(2, 1)
```

## 8 puzzle problem solve

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 0 | 7 | 8 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 0 |

*# Example usage*

`initial = [1, 2, 3, 4, 5, 6, 0, 7, 8]`

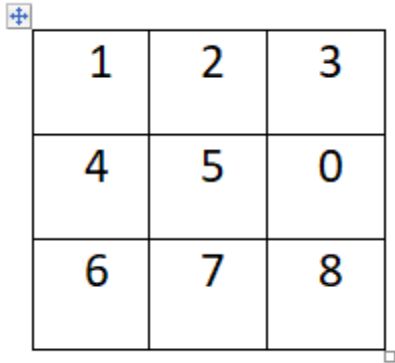
`goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]`

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 0 | 4 | 5 |
| 6 | 7 | 8 |

|   |   |   |
|---|---|---|
| 1 | 2 | 0 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

```
def possible_moves(board):  
    moves = []  
    empty_index = board.index(0)  
    if empty_index % 3 > 0: # Move left  
        moves.append(-1)  
    if empty_index % 3 < 2: # Move right  
        moves.append(1)  
    if empty_index > 2: # Move up  
        moves.append(-3)  
    if empty_index < 6: # Move down  
        moves.append(3)  
    return moves
```

```
def manhattan_distance(board, goal):  
    distance = 0  
    for i in range(1, 9):  
        x1, y1 = divmod(board.index(i), 3)  
        x2, y2 = divmod(goal.index(i), 3)  
        distance += abs(x1 - x2) + abs(y1 - y2)  
    return distance
```



|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 0 |
| 6 | 7 | 8 |

```
def apply_move(board, move):  
    new_board = board[:]  
    empty_index = new_board.index(0)  
    new_index = empty_index + move  
    new_board[empty_index], new_board[new_index] = new_board[new_index], new_board[empty_index]  
    return new_board
```

```
def a_star_search(initial_state, goal_state):
    explored = set()
    priority_queue = []
    initial = PuzzleState(initial_state, None, None, 0, manhattan_distance(initial_state, goal_state))
    heapq.heappush(priority_queue, initial)

    while priority_queue:
        current_state = heapq.heappop(priority_queue)
        explored.add(tuple(current_state.board))

        if current_state.board == goal_state:
            return current_state

        for move in possible_moves(current_state.board):
            new_board = apply_move(current_state.board, move)
            if tuple(new_board) not in explored:
                new_state = PuzzleState(new_board, current_state, move, current_state.depth + 1, current_state.depth + 1 + manhattan_distance(new_bo
                heapq.heappush(priority_queue, new_state)

    return None
```

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 0 |
| 6 | 7 | 8 |

```
def possible_moves(board):  
    moves = []  
    empty_index = board.index(0)  
    if empty_index % 3 > 0: # Move left  
        moves.append(-1)  
    if empty_index % 3 < 2: # Move right  
        moves.append(1)  
    if empty_index > 2: # Move up  
        moves.append(-3)  
    if empty_index < 6: # Move down  
        moves.append(3)  
    return moves
```